Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**08**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Develop a program to provide synchronization among the 5 philosophers in Dining Philosophers problem using semaphore.
Theory	The Dining Philosophers Problem is a fundamental example used to illustrate process synchronization and resource sharing in operating systems. It consists of five philosophers sitting around a circular dining table, where each philosopher alternates between two states — thinking and eating. Between every two philosophers, there is a single fork, and a philosopher needs both the left and right forks to eat. The main challenge of this problem is to design a system where no two philosophers can use the same fork at the same time (ensuring mutual exclusion), while also preventing deadlock (when all philosophers pick up one fork and wait forever for the other) and starvation (when some philosophers never get a chance to eat). To handle this synchronization issue, semaphores are introduced as a coordination mechanism. A semaphore is a variable that controls access to shared resources through two atomic operations: **wait()** (also known as P or down) and **signal()** (also known as V or up). Each fork is represented as a semaphore initialized to 1, meaning it is available for use. When a philosopher wants to eat, they perform a wait() operation on both adjacent semaphores, ensuring that both forks are available before proceeding. Once done eating, the philosopher performs a signal() operation on both forks, making them available for others. Using semaphores ensures that only one philosopher can hold a fork at a time, preventing race conditions. Various strategies can also be used to avoid deadlock, such as allowing only four philosophers to sit at the table simultaneously, or enforcing an order where philosophers pick up one fork at a time following specific rules. The use of semaphores thus provides proper synchronization

	among the philosophers, ensuring safe, fair, and efficient access to shared resources without conflicts or indefinite waiting.
Procedure and Execution	

```
//sheikh wasimuddin 23071044 CSEIOT 64
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>

#define N 5
#define CYCLES 3

#define THINKING 0
#define HUNGRY 1
#define EATING 2

#define LEFT (ph_num + 4) % N
#define RIGHT (ph_num + 1) % N

sem_t mutex;
sem_t S[N];
int state[N];

void test(int ph_num)
{
   if (state[ph_num] == HUNGRY &&
      state[LEFT] != EATING &&
      state[RIGHT] != EATING)
   {
      state[ph_num] = EATING;
      sleep(1);
      printf("Philosopher %d takes forks %d and %d\n", ph_num + 1,
LEFT + 1, ph_num + 1);
      printf("Philosopher %d is Eating\n", ph_num + 1);
      sem_post(&S[ph_num]);
   }
}

void take_fork(int ph_num)
{
```

```
    sem_wait(&mutex);
    state[ph_num] = HUNGRY;
    printf("Philosopher %d is Hungry\n", ph_num + 1);

    test(ph_num);

    sem_post(&mutex);
    sem_wait(&S[ph_num]);
    sleep(1);
}

void put_fork(int ph_num)
{
    sem_wait(&mutex);
    state[ph_num] = THINKING;
    printf("Philosopher %d putting forks %d and %d down\n", ph_num + 1,
LEFT + 1, ph_num + 1);
    printf("Philosopher %d is Thinking\n", ph_num + 1);

    test(LEFT);
    test(RIGHT);

    sem_post(&mutex);
}

void* philosopher(void* num)
{
    int ph_num = (intptr_t)num;

    for (int i = 0; i < CYCLES; i++)
    {
        sleep(1);
        take_fork(ph_num);
        sleep(2);
        put_fork(ph_num);
    }

    printf("Philosopher %d has finished eating and leaves the table.\n",
ph_num + 1);
    return NULL;
}

int main()
{
```

```
    pthread_t thread_id[N];
    sem_init(&mutex, 0, 1);
    for (int i = 0; i < N; i++)
        sem_init(&S[i], 0, 0);

    for (int i = 0; i < N; i++)
    {
        pthread_create(&thread_id[i], NULL, philosopher,
(void*)(intptr_t)i);
        printf("Philosopher %d is Thinking\n", i + 1);
    }

    for (int i = 0; i < N; i++)
        pthread_join(thread_id[i], NULL);

    printf("\nAll philosophers have finished their meals. Dinner is
over!\n");
    return 0;
}
```

Output:

main.c

```
//sheikh wasimuddin 23071044 CSEIOT 64
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdint.h>

#define N 5
#define CYCLES 3

#define THINKING 0
#define HUNGRY 1
#define EATING 2

#define LEFT (ph_num + 4) % N
#define RIGHT (ph_num + 1) % N

sem_t mutex;
sem_t S[N];
int state[N];

void test(int ph_num)
{
    if (state[ph_num] == HUNGRY &&
        state[LEFT] != EATING &&
        state[RIGHT] != EATING)
    {
        state[ph_num] = EATING;
        sleep(1);
        printf("Philosopher %d takes forks %d and %d\n", ph_num + 1, LEFT + 1, ph_num + 1);
        printf("Philosopher %d is Eating\n", ph_num + 1);
        sem_post(&S[ph_num]);
```

```
Philosopher 1 is Thinking
Philosopher 2 is Thinking
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 5 is Thinking
Philosopher 2 is Hungry
Philosopher 2 takes forks 1 and 2
Philosopher 2 is Eating
Philosopher 1 is Hungry
Philosopher 3 is Hungry
Philosopher 4 is Hungry
Philosopher 4 takes forks 3 and 4
Philosopher 4 is Eating
Philosopher 5 is Hungry
Philosopher 2 putting forks 1 and 2 down
Philosopher 2 is Thinking
Philosopher 1 takes forks 5 and 1
Philosopher 1 is Eating
Philosopher 4 putting forks 3 and 4 down
Philosopher 4 is Thinking
Philosopher 3 takes forks 2 and 3
Philosopher 3 is Eating
Philosopher 2 is Hungry
Philosopher 4 is Hungry
Philosopher 1 putting forks 5 and 1 down
Philosopher 1 is Thinking
Philosopher 5 takes forks 4 and 5
Philosopher 5 is Eating
Philosopher 3 putting forks 2 and 3 down
Philosopher 3 is Thinking
Philosopher 2 takes forks 1 and 2
```

Output Analysis	When the program is executed, all five philosophers start in the **thinking state**, as indicated by the initial messages such as "Philosopher 1 is Thinking," "Philosopher 2 is Thinking," and so on. Each philosopher then alternates between the states of **thinking**, **hungry**, and **eating**, demonstrating how synchronization is managed among concurrent threads. When a philosopher becomes hungry, they try to pick up their left and right forks. If both forks are available and neither of the neighboring philosophers is eating, the philosopher successfully takes the forks and enters the **eating** state. This is shown by outputs like "Philosopher 3 takes forks 2 and 3" and "Philosopher 3 is Eating." Once a philosopher finishes eating, they put both forks down and return to the **thinking** state, allowing their neighbors to start eating if possible. This prevents any philosopher from monopolizing the forks and ensures that everyone gets a fair chance to eat. The semaphores used in the program (mutex and S[N]) ensure **mutual exclusion** and **proper synchronization**—only one philosopher can access shared resources at a time, and no two adjacent philosophers can eat simultaneously. After completing the predefined number of eating cycles (for example, three), each philosopher finishes their routine, shown by the message "Philosopher X has finished eating and leaves the table." Finally, when all philosophers have completed their meals, the program terminates with the message "All philosophers have finished their meals. Dinner is over!" This confirms that the program successfully prevents **deadlock** and **starvation**, maintaining a balanced and coordinated execution of concurrent processes.
Conclusion	The Dining Philosophers program successfully demonstrates the concept of **process synchronization** using **threads and semaphores**. It ensures that multiple philosophers (processes) can share limited resources (forks) without causing **deadlock** or **starvation**. By using a global mutex for mutual exclusion and individual semaphores for each philosopher, the program maintains proper coordination, allowing only safe and orderly access to shared resources. Each philosopher alternates between thinking, eating, and putting down forks, showing how concurrent processes can operate harmoniously. Overall, the program effectively illustrates how synchronization techniques can prevent conflicts and ensure fairness in multi-process systems.
Plag Report (Similarity index < 12%)	
Date	27/10/2025